Hladový algoritmus

- Hladové algoritmy na každém kroku dělají lokálně optimální volby k řešení problému.
  - Mohou a nemusí najít globální optimum.
- Příklady
  - Rozměňování mincí
    - Chceme sestavit danou sumu při využití minima mincí
  - Problém výběru aktivit
    - Pokud máme množinu aktivit s časy začátku a konce
    - Problémem je vybrat maximální počet aktivit, které může vykonat jedna osoba nebo stroj za předpokladu, že osoba může pracovat pouze na jedné aktivitě najednou.
- Hladové algoritmy představují jednoduché a často účinné řešení. Ovšem v řadě případů nedosáhnou optima nebo není garantováno, že zadání vyřeší za všech podmínek
- Rozměňování mincí
  -
```python
coins: list[int] = [50, 20, 10, 5, 2, 1]
available = [10, 3, 5, 10, 10, 20]
def coin_changing(amount: int, available: list[int]):
    # Inicializace seznamu počtu použitých mincí
    counts = [0] * len(coins)
    available = available.copy()
    # Iterace přes mince v sestupném pořadí
    for i in range(len(coins)):
        while amount >= coins[i] and available[i] > 0:
            amount-= coins[i]
            available[i]-= 1
            counts[i] += 1
    return None if amount > 0 else counts

finalCount = (coin_changing(192,available))

for i in range(len(coins)):
    print(f"{coins[i]}: {finalCount[i]}X")
```
- Rozměňování mincí – Backtrackingem
  - Rekurzivní backtracking = konkrétní technika (backtracking), která je vyjádřena pomocí rekurze
    - backtracking algoritmy mají často rekurzivní podstatu
  -
```python
def coin_changing(amount: int, available: list[int]) -> list[int]:
    # Inicializace seznamu počtu použitých mincí
    counts = [0] * len(coins)

    # Rekurzivní funkce pro výpočet počtu použitých mincí
    def backtrack(amount_left: int, coin_index: int) -> bool:
        nonlocal counts
        if amount_left == 0:
            return True
        # Pokud jsme rozměnili celou částku, vracíme True
        # Pokud jsme prošli všechny mince nebo překročili hodnotu, vracíme False
        if coin_index >= len(coins) or amount_left < 0:
            return False
        # Iterace přes možné počty aktuální mince
        for count in reversed(range(available[coin_index] + 1)):
            if backtrack(amount_left - count * coins[coin_index], coin_index + 1):
                counts[coin_index] = count
                return True  # Pokud jsme našli platné řešení, vracíme True
        return False

    # Spuštění rekurzivní funkce
    success = backtrack(amount, 0)
    return counts if success else None


coins: list[int] = [50, 20, 10, 5, 2, 1]
# available = [1, 10, 10, 10, 10, 50]
available = [1, 3, 0, 1, 1, 100]
# available = [1, 3, 0, 1, 1, 0]
counts = coin_changing(67, available)
if counts is None:
    print("Směna se nepodařila!")
else:
    # Výpis počtu použitých mincí
    print("Počet použitých mincí:")
    for i in range(len(coins)):
        if counts[i] > 0:
            print(f"{coins[i]} Kč: {counts[i]}x")
```

- Časová složitost
    - n = počet různých nominálních hodnot mincí
    - m = maximální počet dostupných mincí pro jakoukoliv nominální hodnotu
    - Časová složitost v nejhorším případě: $T(n,m) = m^n + nm$
        - $\Theta(m^n)$

Rozděl a panuj

- Princip
    - Rozděl problém na několik podproblémů (stejného druhu).
    - Vyřeš (panuj) každý podproblém rekurzivně.
    - Kombinuj řešení podproblémů do celkového řešení.
- Nejčastější použití.
    - Rozděl problém velikosti n na dva podproblémy velikosti n/2: log2(n) řešení.
    - Vyřeš (opanuj) oba podproblémy rekurzivně.
    - Kombinuj dva výsledky do celkového řešení. Časová složitost $\Theta(n)$.
- Složitost pro náš případ:
    - $\Theta(n \cdot \log(n))$
- Mergesort (řazení slučováním)
    - Základní myšlenka
        - Dvě seřazené posloupnosti lze sloučit do jedné v lineárním čase
    - Vnější řazení: potřebujeme paměťový prostor navíc!
    - Postup
        1. procházíme obě posloupnosti současně
            - držíme aktuální index pro obě
        2. do výsledné posloupnosti zapíšeme vždy menší z dvou aktuálních prvků
        3. v příslušné posloupnosti se posuneme na další prvek
    - Rekurzivní postup
        1. je-li vstupní pole jednoprvkové, pak se rovnou vrátí – je seřazené
        2. v opačném případě se rozdělí na dvě menší pole A, B
        3. A i B se rekurzivním voláním seřadí
        4. metodou merge se seřazená pole spojí a výsledek se vrátí
    - Složitost
        - $\Theta(n \log(n))$
        - Paměťová složitost
            - $\Theta(n)$
            - nutná pro uložení seřazené posloupnosti,
    - Příklad: pole: [2,8,4,1]
        - rozdělíme na [2,8] a [4,1]
        - rozdělíme [2,8] na [2] a [8] a [4,1] na [4] a [1]
        - čísla srovnáme a výsledkem je [2,8] a [1,4]
        - porovnáme
            - první prvky 2 a 1 → menší je 1, přidáme do výsledku
            - pak 2 a 4 → menší je 2, přidáme
            - porovnáme 8 a 4 → menší je 4, přidáme
            - zbývá 8, přidáme nakonec
        - Výsledné spojení: [1, 2, 4, 8]

- Implementace

```python
def merge_sort(array: list) -> list:
    # Pokud je seznam prázdný nebo má jeden prvek, vrátí ho
    if len(array) <= 1:
        return array
    # Najde střední bod seznamu a rozdělí ho na dva podseznamy
    mid = len(array) // 2
    left = array[:mid]
    right = array[mid:]
    # Rekurzivně seřadí levý a pravý podseznam
    left = merge_sort(left)
    right = merge_sort(right)
    # Sloučí seřazené levé a pravé podseznamy
    array = merge(left, right)
    # Vrátí seřazený seznam
    return array

def merge(left: list, right: list) -> list:
    # Vytvoří seznam správné velikosti pro uložení výsledku
    result = [None] * (len(left) + len(right))
    i, j, k = (0, 0, 0)  # Ukazatel na levý, pravý seznam a na výsledek

    # Opakuj, dokud nebudou oba seznamy vyčerpány
    while i < len(left) and j < len(right):
        # Porovnej aktuální prvky obou seznamů a přidej menší do výsledku
        if left[i] <= right[j]:
            result[k] = left[i]
            i += 1
        else:
            result[k] = right[j]
            j += 1
        k += 1

    # Přidejte zbývající prvky z levého nebo pravého seznamu do výsledku
    while i < len(left):
        result[k] = left[i]
        i += 1
        k += 1

    while j < len(right):
        result[k] = right[j]
        j += 1
        k += 1

    # Vrátí výsledný seřazený seznam
    return result

# Příklad použití:
arr = [38, 27, 43, 3, 9, 82, 10]
sorted_arr = merge_sort(arr)
print("Seřazené pole:", sorted_arr)
```